

# Cohort — Quick Reference

Commands & skills, ordered for day-to-day use. Slash commands run in your IDE; cohort ... in the terminal. Generated from canonical.

## BUILD & SHIP · DAILY

- /plan** Break work into small verifiable tasks with acceptance criteria and dependency ordering
- /build** Implement the next task incrementally — build, test, verify, commit
- /test** Run TDD workflow — write failing tests, implement, verify. For bugs, use the Prove-It pattern.
- /goal** Drive a GitHub issue to a draft PR — build, then an independent judge verifies each criterion (max 3 rounds)
- /code-review** Conduct a five-axis code review — correctness, readability, architecture, security, performance
- /code-simplify** Simplify code for clarity and maintainability — reduce complexity without changing behavior
- /spec** Start spec-driven development — write a structured specification before writing code
- /ship** Run the pre-launch checklist via parallel fan-out to specialist personas, then synthesize a go/no-go decision

## MULTI-VENDOR · ORCHESTRATE & REVIEW

- /crew** Cross-vendor build — a coordinator routes work to the cheapest capable model from any vendor; Claude subagents write, external engines propose gated patches
- /scout** Multi-vendor adversarial review — fan disjoint areas to Claude tiers, ChatGPT, and Grok, cross-examine over two rounds, synthesize with convergence
- /ratchet** Metric-gated autonomous optimization loop — propose, evaluate, keep the gain or revert, in a throwaway worktree; you review the staircase and merge
- /consult-gpt** Get a second opinion from ChatGPT (via the OpenAI Codex CLI, read-only) on a hard problem — advisory, cross-examined, never executed blindly
- /consult-grok** Get a second opinion from Grok (via xAI's API, direct) on a hard problem — advisory, cross-examined, never executed blindly

## EXTERNAL ENGINES · COHORT ENGINE ...

- consult <e> --tier** One-shot advisory, read-only. Tier: cheap | flagship | reasoning.
- review <e>** Read-only agentic explore loop — gated per read, transcript recorded.
- propose <e> --agentic** Engine proposes a patch; Cohort applies it in a worktree behind the gates. Grok's write path.
- work gpt** Codex edits natively in its OS sandbox, confined to a throwaway worktree.
- ratchet <gpt|grok>** The /ratchet loop from the CLI. --evaluator "... " --budget N.

The line: Claude writes your tree; external engines only ever touch a throwaway worktree — gated proposal (Grok) or OS-sandbox (Codex). You review & merge.

## THE OFFICE

- ChiefOfStaff** Ask first for cross-functional questions — it names the right specialist(s) to consult.
- /office-setup** Interview the user and tailor the global office — wiring, context, and custom agents.
- /project-setup** Interview the team about this repo — fill project context and scaffold tailored specialists.

## MANAGE COHORT · COHORT ...

- dashboard** Local web UI (mission control) at 127.0.0.1:8787.
- /update** Update Cohort to the latest upstream version and recompile your IDEs.
- recompile --ide X** Recompile the office into an IDE (claude/codex/cursor/copilot).
- status** Read-only view of the install.
- my-office sync** Sync your personal layer to/from a git remote.
- my-office review.approve** Review & approve quarantined pulled artifacts.
- office review.approve** Review & approve office-layer quarantine.
- /snapshot** Capture a session snapshot for this repo's Cohort context, filled from this conversation.
- /feedback** Rate an office agent or command from inside the IDE (fuels the improvement loop).
- distill** Compound recent sessions + feedback into project memory.

## SKILLS · AUTO-FIRE WHEN RELEVANT

- office-guide** How to work with the Cohort agentic office — routing, the daily loop, health checks.
- adversarial-review** How to run a multi-vendor adversarial review — disjoint areas, model/vendor routing, two-round refutation, convergence synthesis (the /scout method).
- karpthy-discipline** Four sharp coding rules (Karpthy) — think before coding, simplicity first, surgical changes, goal-driven — as measurable tests, complementing fable-mode's process gates.

## MORE COMMANDS

- /audit** Recurring deep adversarial audit of the application and the business that ships it — rotates dimensions and subsystems, keeps a coverage ledger, and sweeps the critical path and go-to-market every run
- /barney** Explain something Barney style — so simple, so step-by-step, that nobody can get it wrong
- /brainstorm** Recurring generative sweep for new opportunities and ideas — the creative sibling of /audit. Diverges wide across creative lenses, converges on the strongest, turns survivors into pursuit plans, and keeps an idea ledger so it compounds run over run and never re-generates a dead idea.